

Comandos utilizados no Git e GitHub

Atividade individual

Nome: Higor Xavier Galhardo \ **Matrícula:** 36019348 \ **Data:** 24/09/2026 \ **Horário limite:**
21:00

Sumário

1	Introdução	1
2	Comandos fundamentais do Git	1
3	Branches e integração de alterações	2
4	Comandos para sincronização com o GitHub	2
5	Principais ações no GitHub	3
6	Boas práticas	3
7	Conclusão	3
8	Referências	3

1 Introdução

Git é um sistema de controle de versões distribuído. Ele registra alterações em arquivos, permite recuperar versões anteriores e facilita o trabalho em equipe. GitHub é uma plataforma on-line que hospeda repositórios Git e oferece recursos de colaboração, como branches, pull requests, issues e revisão de código. Os comandos abaixo estão organizados de acordo com as tarefas mais comuns: configurar o ambiente, criar ou obter um repositório, registrar alterações, trabalhar com branches e sincronizar o trabalho com o GitHub.

2 Comandos fundamentais do Git

Comando	Função
<code>git --version</code>	Exibe a versão instalada do Git e ajuda a confirmar se a ferramenta está disponível.
<code>git config --global user.name "Nome"</code>	Define o nome que será registrado nos commits do usuário.
<code>git config --global user.email "email"</code>	Define o endereço de e-mail associado aos commits.
<code>git config --list</code>	Lista as configurações atualmente aplicadas.
<code>git init</code>	Inicializa um novo repositório Git no diretório atual, criando a pasta oculta <code>.git</code> .
<code>git clone URL</code>	Copia um repositório remoto para o computador local.
<code>git status</code>	Mostra arquivos modificados, não rastreados e prontos para commit.
<code>git add arquivo</code>	Coloca um arquivo na área de preparação, também chamada de staging area.
<code>git add .</code>	Coloca na área de preparação as alterações encontradas no diretório atual.
<code>git commit -m "mensagem"</code>	Cria um registro permanente das alterações preparadas, acompanhado de uma mensagem.
<code>git log</code>	Exibe o histórico de commits do repositório.
<code>git diff</code>	Apresenta as diferenças entre o estado atual dos arquivos e a versão registrada.
<code>git restore arquivo</code>	Descarta alterações locais ainda não registradas, quando usado com cuidado.
<code>git rm arquivo</code>	Remove um arquivo do projeto e registra essa remoção para o próximo commit.

3 Branches e integração de alterações

Uma branch é uma linha independente de desenvolvimento. Ela permite trabalhar em uma funcionalidade sem alterar imediatamente a branch principal.

Comando	Função
<code>git branch</code>	Lista as branches locais.
<code>git branch nome</code>	Cria uma nova branch com o nome indicado.
<code>git switch nome</code>	Muda para uma branch existente.
<code>git switch -c nome</code>	Cria uma branch e muda para ela imediatamente.
<code>git checkout nome</code>	Forma tradicional de mudar para uma branch ou restaurar arquivos.
<code>git merge nome</code>	Integra na branch atual os commits da branch indicada.
<code>git rebase nome</code>	Reposiciona commits sobre outra base para reorganizar o histórico; deve ser usado com atenção em branches compartilhadas.
<code>git branch -d nome</code>	Exclui uma branch local que já foi integrada.

4 Comandos para sincronização com o GitHub

O remoto é o endereço do repositório hospedado no GitHub. Normalmente ele recebe o nome `origin`. Os comandos seguintes conectam o repositório local ao remoto e mantêm os históricos sincronizados.

Comando	Função
<code>git remote -v</code>	Exibe os endereços dos repositórios remotos configurados.
<code>git remote add origin URL</code>	Adiciona um repositório remoto e o identifica como <code>origin</code> .
<code>git push -u origin main</code>	Envia a branch <code>main</code> ao GitHub e estabelece o vínculo padrão entre a branch local e a remota.
<code>git push</code>	Envia para o remoto os commits locais que ainda não foram publicados.
<code>git fetch</code>	Baixa referências e commits do remoto sem alterar automaticamente os arquivos da branch atual.
<code>git pull</code>	Baixa alterações do remoto e tenta integrá-las à branch atual.
<code>git pull --rebase</code>	Baixa alterações e reaplica os commits locais sobre a versão mais recente do remoto.
<code>git push origin --delete nome</code>	Exclui uma branch do repositório remoto, desde que o usuário tenha permissão.

5 Principais ações no GitHub

Além dos comandos executados no terminal, o GitHub oferece ações pela interface web. Criar um repositório organiza o projeto em um espaço próprio. Um commit registra uma alteração. Uma branch permite desenvolver uma tarefa separadamente. Um pull request solicita a revisão e a integração de alterações em outra branch. Uma issue registra uma tarefa, dúvida ou problema. O arquivo README.md apresenta a finalidade e as instruções básicas do projeto.

Para enviar um projeto local para um repositório novo, uma sequência comum é:

```
git init\git add .\git commit -m "Primeiro commit"\git branch -M main\git remote  
add origin URL_DO_REPOSITORIO\git push -u origin main
```

Essa sequência inicializa o projeto, prepara os arquivos, cria o primeiro registro, define a branch principal, conecta o projeto ao GitHub e publica os commits.

6 Boas práticas

Antes de executar um comando destrutivo, é importante conferir o resultado de `git status`. Mensagens de commit devem ser curtas e descrever claramente a alteração realizada. O arquivo `.gitignore` deve impedir o envio de senhas, chaves de API, arquivos temporários e dependências que podem ser recriadas. Alterações devem ser sincronizadas com frequência, e branches devem ter nomes objetivos, como `feature/login` ou `fix/correcao-menu`.

Também é recomendável revisar o histórico antes de fazer um push, manter a branch principal estável e usar pull requests quando houver trabalho colaborativo. Com essas práticas, o Git registra a evolução do projeto e o GitHub facilita a colaboração e a publicação do código.

7 Conclusão

Os comandos do Git formam um fluxo de trabalho completo: configurar a identidade, criar ou clonar o repositório, verificar alterações, preparar arquivos, registrar commits, criar branches e sincronizar o histórico. O GitHub complementa esse fluxo ao hospedar o repositório e oferecer recursos de colaboração. O domínio de comandos como `status`, `add`, `commit`, `branch`, `merge`, `pull` e `push` é essencial para controlar versões de forma organizada e segura.

8 Referências

- Git SCM. **Git Documentation**. <https://git-scm.com/docs>
- GitHub Docs. **GitHub documentation**. <https://docs.github.com/en/get-started/learning-about-github>
- GitHub Docs. **About Git**. <https://docs.github.com/en/get-started/learning-about-github/github-glossary#git>